Advancing eBPF JIT Security through Hardware Capabilities

Barna Ifkovics
b.ifkovics@student.utwente.nl
Supervisor: dr.ir. A. Continella
Daily supervisor: Mattia Napoli

Abstract

The security of modern operating system kernels increasingly depends on the safe execution of untrusted extensions. This transition is best exemplified by the conception of the extended Berkeley Packet Filter, serving as the de facto standard for Linux kernel modularity. Despite its relevance, eBPF's reliance on complex in-kernel static verification is reaching its analytical limits. The fragility of software-centric security measures is highlighted by numerous CVEs rooted in range analysis failures and type confusion. In this paper, we explore the potential of hardware-enforced protection using the Capability Hardware Enhanced RISC Instructions architecture. By implementing a just-in-time compiler for the Arm Morello prototype, we demonstrate how traditional pointers can be replaced with unforgeable 128-bit capabilities to enforce spatial and temporal memory safety. Our work proposes that offloading security enforcement from software verifiers to the underlying microarchitecture provides a robust path for future kernel extensibility, while significantly reducing the kernel's trusted computing base. We hypothesize that shifting the security burden to CHERI hardware reduces verifier complexity while providing robust, hardware-rooted protection against memory errors. While this transition likely incurs a performance overhead, we argue that it is a necessary trade-off for achieving resilience against evolving eBPF exploitation techniques.

UNIVERSITY OF TWENTE.

1 Introduction

The evolution of the Linux kernel from a static, monolithic entity toward a dynamically extensible platform represents a significant departure from traditional operating system design philosophies [1]. Historically, extending kernel functionality required either the development of loadable kernel modules (LKMs) [2] or direct modifications to the source code, both of which present substantial risks to system security. While direct source code modifications necessitate full kernel recompilation and impose significant maintenance overhead, LKMs operate without an architectural safety barrier [3]. Consequently, a single memory error or logic flaw in such extensions can compromise kernel integrity.

The extended Berkeley Packet Filter (eBPF) emerged as a solution to this architectural problem of balancing safety, performance, and programmability. It employs a register-based virtual machine with a custom instruction set architecture [4] that is just-in-time-compiled (JIT) to native code, enabling the execution of sandboxed programs at near-native performance within the privileged context of the kernel. By decoupling the extension logic from the core kernel source, eBPF allows developers to infuse custom logic at predefined points, without compromising system integrity. Its integration into the Linux kernel is not merely an additive feature but a fundamental shift in how the kernel mediates between hardware resources and user-provided logic.

eBPF was initially designed for filtering network packets [5] but grew beyond its limited role and evolved into a versatile, in-kernel framework for general-purpose program execution, supporting functionality from dynamic tracing [6] to advanced security orchestration [7]. By utilizing a blend of static analysis carried out by a *verifier* [8], and subsequent JIT compilation, with the support of an *interpreter*, it promises predictable execution and memory safety at near-native speeds. Since the sole duty of its JIT compiler [9] is to convert `bpf` bytecode, a RISC-like intermediate

representation, into native instructions for direct execution, all unsupported or unsafe logic must be filtered by the verifier. To maintain kernel cohesion, this filtering process relies on instruction depth counters and strictly defined memory regions. However, as eBPF logic grows increasingly intricate to meet the demands of cloud-scale environments [10], these static analysis techniques are hitting their breaking point. The verifier faces a fundamental trade-off: it either over-approximates safety, leading to the rejection of benign but complex programs, or suffers from under-approximation that overlooks subtle logic flaws. This growing tension between the need for complex, high-performance modularity and the absolute requirement for kernel safety has turned the verification process into the primary bottleneck for future innovation.

Recognizing this limitation, new research [11] has pivoted toward hardening the existing verification pipeline by employing *abstract interpretation*. Their prototype PREVAIL employs a formal framework that approximates program semantics precisely to verify safety properties without exhaustive execution. While this idea improves reliability, software-only verification remains constrained by the battle between analytical precision and computational overhead, which often results in increased verifier complexity. Consequently, this work proposes an alternative trajectory: offloading security enforcement to hardware-level features to achieve deterministic memory safety through architectural primitives.

A prominent path is dynamic sandboxing [12], which bypasses the rigid constraints of the verifier by instrumenting JIT-compiled code with runtime checks. However, this approach remains inadequate for high-performance environments as the recurrent insertion of software-defined guards introduces substantial runtime overhead that can become prohibitive. Simultaneously, the upstream Linux kernel has begun integrating hardware-assisted defenses like pointer authentication and branch target identification [13] on Arm architectures. These features provide a safety net for control-flow integrity, ensuring that even if a logic flaw slips through the verifier, the program cannot easily be coerced into executing unintended paths. Although these mechanisms prevent the redirection of execution flow to arbitrary execution paths, they do not solve data-plane corruption or out-of-bounds memory accesses within eBPF programs. Further explorations [14] have looked into the Memory Tagging Extension (MTE) [15] as a way to provide memory safety. By tagging memory regions at the hardware level, the hardware can intercept runtime violations that exceed the verifier's analytical limits, and potentially facilitate intricate logic that static analysis would otherwise reject as unprovable [16]. However, while technologies like MTE offer probabilistic protection through 4-bit tags, they remain vulnerable to speculative execution attacks [17] and fail to address the fundamental issue of pointer provenance, rendering them reactive measures built on top of a legacy memory model. A more radical shift is emerging that could reinvent the relationship between the verifier and the kernel: the move toward capability-based architectures. Among these, the Capability Hardware Enhanced RISC Instructions (CHERI) [18] architecture presents one of the most mature and researched implementations of this paradigm. This maturity is evidenced by over fifteen years of development and physical silicon realization, such as the Arm Morello prototype [19]. It also supports a complete software ecosystem, including CheriBSD [20] and a mature LLVM-based toolchain [21], which demonstrates its practical viability.

We propose a conceptual shift in the eBPF safety model: offloading safety enforcement from the software verifier to hardware capability primitives. Integrating eBPF with the CHERI architecture provides a secure environment for kernel-level programmability through fine-grained memory safety [22]. Adopting capability-based addressing facilitates a reduction in runtime overhead by replacing heavy software instrumentation with native hardware checks. This paradigm allows for the enforcement of strict temporal and spatial safety boundaries at the instruction level, effectively neutralizing common exploit vectors such as out-of-bounds access. Consequently, it delegates the security burden from computationally expensive software verifiers to the CHERI microarchitecture, directly addressing the safety-performance trade-offs in existing kernel extension models [23]. Such hardware-enforced isolation significantly reduces reliance on the eBPF verifier's completeness and simplifies the security proofs [24] required for kernel-resident code. We evaluate this approach using the Morello prototype, an Arm-based system-on-chip designed to demonstrate the viability of CHERI at scale. The integration of per-page capability load barriers within Morello supports high-performance temporal safety mechanisms, such as Cornucopia Reloaded[25], to secure dynamically allocated heap memory used by eBPF programs.

To realize this model, we augment the AArch64 eBPF JIT compiler to leverage Arm Morello capability hardware. Our proposed implementation remaps eBPF registers to 128-bit CHERI capability registers and replaces standard memory instructions with hardware-validated primitives, effectively delegating safety enforcement

from the software verifier to the microarchitecture. This transition includes developing capability shims for kernel helper functions and enforcing 16-byte stack alignment to maintain pointer provenance. We evaluate the system by porting high-severity CVEs [26, 27] to Morello, demonstrating how hardware-rooted enforcement neutralizes vulnerabilities that bypass current static analysis and quantifying the associated performance-security trade-offs.

The rest of this paper is organized as follows: section 2 provides background on eBPF and CHERI; section 3 reviews related work; section 4 outlines the problem; section 5 describes our methodology; section 6 lays out a research timeline; and section 7 concludes.

2 Background

eBPF [28] offers a programmable VM for executing user-authored code safely within the Linux kernel. Unlike traditional LKMs, which operate under unrestricted kernel privileges, eBPF allows for the execution of user-defined logic at predefined kernel hook points with safety guarantees. The Linux kernel introduces these hooking points at critical locations in its system, and eBPF code is exclusively invoked at specific kernel events. Its ecosystem [29] contains a `bpf` LLVM compiler backend that translates C to a specialised 64-bit RISC bytecode, a static verifier, and a performant runtime comprising both a JIT compiler and an interpreter as illustrated in Figure 1.

2.1 eBPF Virtual Machine Architecture

The eBPF runtime is defined as a 64-bit RISC architecture. It features 11 64-bit registers: ten general purpose registers ranging from R_0 to R_9, and a dedicated read-only frame pointer in register R_{10} for stack access [30]. This design reflects a deliberate effort to mirror the register-rich environments of modern CPUs, enabling a high degree of one-to-one mapping between eBPF bytecode and native machine instructions.

The register allocation strategy is strictly defined by the eBPF calling convention [4], which ensures compatibility with the kernel's internal application binary interface (ABI). R_0 is reserved for the return value of both the eBPF program and any helper function calls. Registers R_1 through R_5 are designed for passing arguments to external functions, while R_6 through R_9 are used as callee-saved registers that must be preserved across function calls. This mapping is critical for the JIT compiler, as it allows the generated native code to adhere to the host architecture's calling conventions without the overhead of complex register shuffling or excessive stack spills.

The increased register width from the classic

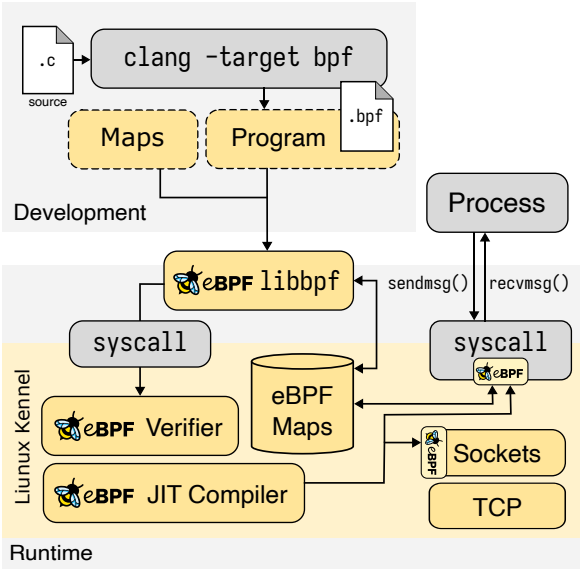

Figure 1: Overview of the eBPF runtime and interaction model.

32-bit to 64-bit [5] enables eBPF to handle kernel-space pointers directly, which is vital for interacting with 64-bit memory addresses and complex data structures. However, the ISA [4] maintains 32-bit sub-registers for backward compatibility and specialized arithmetic operations, mapping them to the lower 32 bits of the native 64-bit registers. This dual-width capability allows the JIT compiler to optimize operations where 32-bit precision is sufficient, potentially reducing the instruction footprint and improving cache locality.

Programs are typically authored in a restricted subset of C and compiled via the `bpf` LLVM backend [31] into a hardware-agnostic bytecode. This intermediate representation (IR) is then loaded into the kernel via the `bpf()` system call, where it must pass a rigorous static verification phase before it is permitted to execute.

2.2 Static Verification

The eBPF verifier is the primary arbiter of kernel integrity and thus the main focus of security evaluations. Its primary task is to guarantee safe execution [8] and accomplishes it by evaluating `bpf` programs against a predetermined set of criteria. Furthermore, it optimizes execution paths using information gathered during the verification process. The verifier exists to prevent memory corruption, information leakage, and anything that might cause the kernel to crash or hang. Since programs are later translated into native machine code and executed in privileged mode, verification is an essential step for kernel security. This two-step static model is a trade-off between security and performance. After a program passes the verifier, there are no expensive runtime checks, therefore eBPF programs can run at native speeds. Any

alternative solution using only a virtual machine or interpreter would introduce unwanted execution-time overhead.

The verifier checks every conceivable state-permutation of a program [1] by performing symbolic execution and data-flow analysis. It models loaded programs as a directed acyclic graph (DAG) of basic blocks, and performs a depth-first search to explore all feasible execution paths. During DAG traversal, the verifier tracks the state of each register, including its type and its potential value ranges. Doing this, it is able to distinguish between branches that are always taken, conditionally taken, and never taken. This tracking is critical for preventing out-of-bounds memory accesses, protecting the kernel address space, and dead code elimination (introduced in version 4.15), which replaces dynamically unreachable branches with `NOP` instructions. When the verifier encounters a conditional branch, it forks the current state to evaluate both paths. To mitigate the state-space explosion inherent in complex programs, the verifier also employs *state pruning* [32]: optimizing verification by caching program states at control-flow junctions and terminating redundant path analysis when current register and stack configurations are equivalent to previously verified states.

The verifier attempts to explore all possible program states, however, to ensure tractable analysis and prevent resource exhaustion, it enforces limits on the number of inspected instructions. Different execution paths accumulate instructions separately towards these limits. Consequently, verification complexity depends not only on program size but also on the number of branching paths. If any of such limits is violated, the verifier rejects the program entirely. The evolution of the limits reflect the growing complexity of eBPF applications. Early kernel versions (pre-v5.2) restricted programs to 4096 instructions and imposed a verification complexity bound of 128 000 inspected instructions [33]. Modern kernels raise both limits to 1 million. Early implementations also forbade loops to prevent non-termination during verification, pushing compilers to depend on loop unrolling, which increases program size and is not always practical. Since version 5.3, bounded loops are supported, enabling the verifier to prove loop termination. However, this requires exploring loop execution paths, significantly increasing the number of inspected instructions and pushing programs closer to the complexity limit.

2.3 Kernel integration

Once an eBPF program has been deemed safe by the verifier, the Linux kernel employs a JIT compiler to transform the hardware-agnostic bytecode into optimized native machine instructions. This process is essential for achieving near-native execution speeds, as it eliminates the dispatch overhead and branch misprediction penalties associated with software-based interpreters. The JIT compiler operates in a multi-pass fashion [9], which is a structural requirement for resolving forward-looking branch offsets and calculating final memory requirements for the executable image.

In the initial pass, the JIT compiler iterates through the validated bytecode to evaluate the size of the generated native code. Since eBPF instructions do not always have a constant-size native equivalent, due to the extensive support of architectures with variable-length instructions like x86-64, this initial pass is an approximation based on worst-case scenarios. During this phase, the compiler also identifies the locations of internal jumps and calls to determine the overall structure of the program. The second pass involves the actual generation of machine code. At this stage, the compiler maps each eBPF opcode to its native pair. For programs utilizing BPF-to-BPF calls [34], a third pass is often required to finalize the relocation of call instructions once the absolute start addresses of all sub-programs are known. This iterative approach ensures that all relative branch offsets are accurate, preventing the execution of unintended or unaligned instruction sequences.

To facilitate seamless integration with the host kernel's execution flow, every JIT-compiled image includes a generated prologue and epilogue. The prologue is responsible for initializing the execution environment by decrementing the stack pointer to allocate local space and pushing callee-saved registers onto the stack to preserve the caller's state. It also sets the frame pointer (R_{10}/`rbp`) to point to the base of the program's stack frame, which is essential for DWARF-based stack unwinding and debugging. The epilogue performs the inverse operations, restoring the preserved registers and the stack pointer before executing a native `ret` instruction. This ensures that when the eBPF program finishes, control returns to the kernel hook in a state identical to what existed before the invocation.

While JIT compilation is the preferred execution path [35] for performance-critical environments, the Linux kernel maintains an interpreter as a fallback mechanism. The interpreter is used when JIT compilation is disabled and on hardware architectures that lack a dedicated JIT backend. The interpreter functions as a large switch-case within the kernel function `___bpf_prog_run()`, where each instruction is parsed and executed by software logic. The performance divergence between the interpreter and the JIT compiler is substantial. Because the interpreter must fetch each instruction, decode its opcode, and dispatch

it via software, it suffers from significant per-instruction overhead and high CPU branch misprediction rates [1]. In contrast, a JIT-compiled program runs at the same speed as natively compiled kernel code.

A critical aspect of the eBPF runtime is its interaction with the broader kernel through *helper functions* [36]. eBPF programs are not permitted to call arbitrary kernel functions; instead they are restricted to a stable API of functions provided by the kernel to perform tasks such as map lookups [37], packet redirection [38], and printing to the trace buffer [39]. This restriction ensures that eBPF programs remain portable across kernel versions and do not accidentally call into unstable or unsafe internal kernel paths. Linking in the eBPF context involves a fixup process during JIT compilation. In the eBPF bytecode, calls to helpers are represented as immediate values corresponding to the helper's unique ID. The JIT compiler resolves these IDs to the absolute memory addresses of the corresponding trusted kernel functions. This process is functionally similar to how procedure linkage table (PLT) stubs are replaced by a dynamic linker in user-space, although it occurs entirely within the kernel's privileged context at load-time.

However, the eBPF mechanism is more direct. While user-space PLT stubs often involve lazy binding and indirect jumps through the global offset table, the eBPF JIT compiler hardcodes the actual 64-bit function address directly into the native instruction stream wherever possible. This suppresses the redirection overhead, but it introduces a challenge: if the kernel or a dynamically loaded extension, which provides the helper, is reloaded, this hardcoded addresses in the JIT-ed image become stale, leading to potential crashes. To mitigate this, modern kernels use *trampolines* [40], which provide a stable indirect jump target, allowing the kernel to update the target address of a helper without re-patching every dependent eBPF program.

2.4 CHERI Capability Architecture

The CHERI [41] project, is a joint initiative between the University of Cambridge and SRI International, pushing for an age old [42], yet fundamentally different take on the hardware-software interface to address its shortcomings at the source. CHERI extends conventional ISAs with unforgeable architectural *capabilities*, providing a deterministic, hardware-enforced foundation for fine-grained memory protection and scalable software compartmentalization.

CHERI is a hybrid architecture because it integrates hardware-enforced capabilities into standard RISC pipelines [43] alongside a conventional memory management unit (MMU). While legacy *pure* capability systems, such as the Burroughs B5000 [44] and the Cambridge CAP computer [45], replaced virtual memory entirely [46], CHERI maintains a standard address space. It augments traditional pointers with architectural metadata to enforce deterministic spatial safety. This hybrid design allows developers to selectively use capabilities for fine-grained protection within specific compartments, while the rest of the system continues to rely on standard page-level protections and the existing C software stack [22].

In CHERI, capabilities are not managed in centralized kernel-resident tables. Instead, they are first-class data types that can be stored both in registers and memory. A capability is essentially a pointer that has been extended with metadata, including data bounds and permissions, and protected by a hardware-managed integrity tag [18]. This design allows the compiler to manage capabilities directly, enabling byte-granularity protection that is, in most cases, thousands of times more precise than the page-granularity protection offered by traditional MMUs [41].

2.4.1 What is a capability?

A CHERI capability is an architectural primitive, that takes up 128 bits on 64-bit platforms and 64 bits on 32-bit platforms [18]. Each capability has an architectural address and compressed metadata, further associated with a 1-bit out-of-band validity tag managed by the hardware [43]. Put more simply, we can think of it as a pointer that has defined permissions for what it is allowed to do. Capabilities comprise of the following detailed components as illustrated in Figure 2:

- **Address**: a virtual address.
- **Bounds**: metadata defining a *base* and a *top* limit. The hardware ensures that for any address A, access is only permitted if: $base \leq A < top$ [43].
- **Permissions**: a mask controlling how the capability can be used, such as prohibiting instruction fetch or restricting the loading and storing of other capabilities [41].
- **Object Type** (*otype*): a field used for sealing. If the otype is not equal to -1, the capability is sealed and cannot be modified or dereferenced. Sealed capabilities act like opaque handles, providing a foundation for mutually distrusting software compartmentalization [22].
- **Validity Tag**: an out-of-band bit tracking capability provenance. If the tag is cleared the capability cannot be used for memory access or instruction fetch [41]. It is important to note, that clearing occurs atomically upon any partial overwrite with non-capability data.

To maintain integrity in memory, valid capabilities must also be naturally aligned to 64-bit or 128-bit boundaries [18]. CHERI utilizes the concentrate compression format [18] to compress metadata fields, exploiting address-bounds redundancy to decrease its memory footprint while supporting the wandering pointer idioms common in legacy C code [43].

Mnemonic	Description
`CGetBase`	Move base address to a GPR
`CGetLen`	Move length to a GPR
`CGetTag`	Move tag bit to a GPR
`CGetPerm`	Move permissions to a GPR
`CGetPCC`	Move the PCC and PC to GPRs
`CIncBase`	Increase base and decrease length
`CSetLen`	Set (restrict) capability length
`CClearTag`	Invalidate capability (clear tag bit)
`CAndPerm`	Restrict permissions via bitwise `&`
`CToPtr`	Generate pointer from capability relative to C_0
`CFromPtr`	Create capability from C_0 and integer offset
`CBTU`	Branch if capability tag is unset
`CBTS`	Branch if capability tag is set
`CLC`	Load capability register
`CSC`	Store capability register
`CL[BHWD][U]`	Load byte, half-word, word or double via capability register
`CS[BHWD]`	Store byte, half-word, word or double via capability register
`CLLD`	Load linked capability
`CSCD`	Store conditional capability
`CJR`	Jump to capability register
`CJALR`	Jump and link to capability register

Table 1: CHERI ISA Capability Extensions.

2.4.2 Principle of Intentional Use

A core guiding principle of CHERI is the principle of intentional use. In conventional architectures, memory access is authorized by an ambient authority [18] and a process owns the right to access any address mapped in its page table, and any integer can be used as a pointer. This leads to confused deputy attacks, where a privileged service like an OS kernel is tricked into misusing its authority when acting on behalf of a less privileged client [29].

CHERI eliminates ambient authority by requiring every memory access to be authorized by a specific capability register explicitly named as an operand [18]. The hardware does not implicitly select a source of authority based on the current address, but the software must provide the token. This forces the programmer or the compiler to state their intent before executing a single instruction, allowing the hardware to verify that the specific capability used is valid, tagged, and encompasses the targeted address [43].

2.4.3 Architectural Rules

While CHERI encompasses a broader set of architectural invariants, this work focuses on three fundamental rules: provenance validity, capability monotonicity, and reachable capability monotonicity. These principles ensure that data authority in the system is always traceable, non-elevating, and strictly contained [18].

Provenance validity is the requirement that a valid capability can only be constructed through a valid capability-aware instruction, and only by deriving it from a pre-existing valid capability [41]. This rule prevents the materialization of pointers from arbitrary data. The hardware enforces this rule via a 1-bit out-of-band *tag* associated with every capability-sized and aligned unit of memory and every capability register [18]. When a capability is moved between a register and memory, the tag follows the data. However, if a standard data-store instruction targets any portion of a capability in memory, the hardware atomically clears the tag [22]. An untagged capability is treated by the processor as mere data and cannot be used for memory access or instruction fetch, thus maintaining the integrity of the authority graph [43].

Capability monotonicity ensures that any instruction generating a new capability by deriving it from an old one can only produce a result with equal or lesser rights [18]. Instructions like `CSetBounds` and `CAndPerm` from Table 1 are the primary mechanisms for rights attenuation. If software attempts to perform a non-monotonic operation, such as increasing the length of a capability or adding a permission bit not already present, the hardware responds by either throwing an exception or, in more recent ISA versions like CHERI-RISC-V and Arm Morello, clearing the tag of the resulting capability [18]. Clearing the tag is often preferred in high-performance pipelines to simplify exception logic and prevent speculative execution side-channels [18].

Reachable capability monotonicity is the third, but higher-order property concerning the whole system authority state. It asserts that during the execution of arbitrary code, the set of reachable capabilities, accessible via registers, memory, and the transitive closure of unsealing and derivation, cannot increase [41]. This property provides the theoretical underpinning for sandboxing: if a process is initialized with a restricted set of capabilities, it is mathematically incapable of escaping its bounds to access other parts of the address space without a controlled domain transition [47].

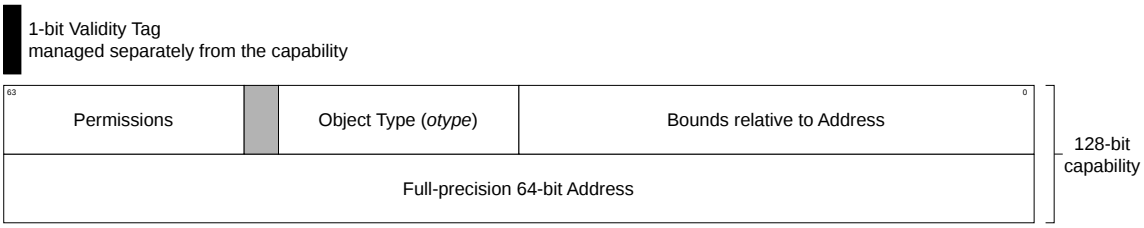

Figure 2: Detailed memory capability layout and bit fields.

2.4.4 Memory Management

It is important to note that unlike other capability systems that replaced the MMU, CHERI works with existing solutions. This relationship between capabilities and the MMU is hierarchical, defined by an evaluate-then-translate sequence [18]. When the CPU executes a memory instruction, the capability hardware first evaluates the provided capability. This evaluation includes tag checking, permission checking, and bounds checking, ensuring that the operation is performed on a valid capability, allowed, and fits the context of the memory bounds. The virtual address passed to the MMU for translation to a physical address only if these capability checks pass [18]. This prioritization ensures that capability violations are caught early and reported as precise capability exceptions, which have a higher priority than page-table faults [18]. This prevent attackers from using the MMU to probe memory layouts or trigger side-channels for addresses they do not have authority to access.

2.4.5 Hybrid Capability

CHERI supports two primary software models, pure and hybrid capability, which can coexist within the same system to provide a path for incremental adoption [41].

In the pure-capability model, such as the one implemented by CheriABI in the CheriBSD operating system, all pointers are represented as architectural capabilities [48]. This includes explicit pointers, which are all language-level pointers, and implied pointers, which are return addresses on the stack, vtable pointers in C++, and the stack pointer itself [41]. This model provides spatial and referential safety, as every object in a program is bound to its specific allocation [41]. However, this is an ABI-disruptive change. Because capabilities are twice the size of standard pointers, the memory layout of structures changes, requiring all libraries to be recompiled [18].

In contrast, the hybrid model is designed for binary compatibility and incremental hardening. In this mode, pointers are standard 64-bit integers by default, which are interpreted relative to a *global ambient capability* [18]. Developers can then selectively use the `__capability` qualifier in C code to promote security-critical pointers to full 128-bit capabilities [22]. This allows legacy code to run unmodified while individual subsystems are hardened [41]. This interoperability is partially managed through the following two key hardware registers.

The default data capability (DDC), which is an architectural register that authorizes all legacy, non-capability-aware load and store instructions [41]. When a legacy instruction accesses an integer address, the hardware implicitly validates it against the bounds and permissions of the DDC [18]. By narrowing the DDC, an operating system can sandbox a legacy binary within a subregion of its address space, enabling compartmentalization without recompilation [22].

And the program counter capability (PCC), which serves as the capability-based equivalent of the program counter, authorizing all instruction fetches [18]. It ensures code execution remains within authorized segments and prevents data from being executed as code [41]. Upon branching, the PCC is updated. For legacy code, the jump is constrained by the current PCC bounds, ensuring that even unmodified binaries adhere to architectural security policies [18].

Beyond these core registers, CHERI incorporates numerous mechanisms to address integration challenges, such as capability sealing and specialized exception handling. These features are omitted here as they are not strictly relevant to the primary discussion of memory safety.

3 Related Work

The following section enumerates prior research addressing the instability and security risks inherent in the eBPF verifier. While initial efforts focused on static analysis hardening, the field has increasingly pivoted toward runtime isolation, ranging from software sandboxing to hardware-enforced domains.

The complexity of the Linux verifier[1] makes it a frequent target for exploitation. Jia et al. [50] argues that the verification-based model is untenable due to the state explosion in symbolic execution and the growing number of unverified ker-

[1]Here we refer to the Linux kernel's eBPF verifier [49].

nel helper functions. Recent attacks, such as the Tailcall Trampoline and JIT-type confusion [51], demonstrate that even verified code can be redirected to achieve arbitrary kernel memory access. To identify such logic bugs, Sun and Su [52] introduced state embedding to detect discrepancies between the verifier's abstract approximations and concrete runtime states.

Efforts to improve verifier precision include the adoption of the Zone abstract domain in PREVAIL [11] to support programs with loops and relational register tracking. Vishwanathan et al. [53] further optimized the verifier by providing optimal operators for the tristate number (*tnum*) domain. To provide formal guarantees, the Agni framework [54] automatically generates SMT-based verification conditions from the kernel's C source to prove the soundness of value-tracking logic. More recently, the eBPF Certificate Framework [24] proposed offloading complex reasoning to user-space satisfiability modulo theory solvers while performing linear-time proof checking within the kernel to achieve symbolic-execution-level precision without kernel-space complexity. Furthermore, the Merlin framework [33] optimizes eBPF bytecode using LLVM passes to merge instructions and reduce runtime overhead by up to 60%, ensuring optimized programs still pass stringent verifier checks.

Recognizing that static checks can be bypassed, researchers have explored software fault isolation. SandBPF [12] utilizes binary rewriting to insert address masking and call trampolines into JIT-ed code, providing a second layer of defense. To address microarchitectural threats, VeriFence [55] modifies the verifier to selectively insert speculation barriers, ensuring that speculative execution paths remain as safe as architectural ones, and reducing the rejection rate of unprivileged `bpf` programs from 54% to nearly zero.

Mainstream hardware features have been leveraged to provide efficient, fine-grained isolation. MOAT [56] utilizes Intel's Memory Protection Keys for Supervisor to partition the kernel and eBPF domains, disabling kernel access when eBPF extensions are active. On Arm architectures, HIVE [57] leverages unprivileged load/store instructions and pointer authentication to restrict eBPF access to kernel objects. SafeBPF [14] further refines this by using Arm MTE to enforce byte-granular spatial memory safety with minimal 0–4% overhead.

Limitations of probabilistic or page-level hardware models have resulted in the exploration of CHERI in recent research. Metzger et al. [58] demonstrate the use of eBPF sandboxes within CHERI-protected driver compartments to handle performance-sensitive interrupts without full kernel privileges. Similarly, CHERI has been applied to automotive networks [59] to deterministic isolation for intrusion detection rules, ensuring rule integrity against IP spoofing and manipulation. These works suggest that adopting the eBPF JIT to CHERI allows the offloading of the verifier's most complex safety checks directly to the hardware.

Despite these implementations, CHERI remains largely unexploited as a primary safety substrate for the eBPF runtime environment. Current research typically utilizes CHERI as an external containment boundary rather than a fundamental component of the JIT compilation process itself. By adapting the eBPF JIT backend to the CHERI ISA, the kernel could offload the verifier's most complex safety checks directly to the hardware.

4 Problem Statement

The Linux kernel eBPF subsystem relies solely on the in-kernel static verifier to ensure that extensions do not violate system integrity [1], however, this verification pipeline is facing a critical bottleneck due to the inherent trade-off between analysis precision and program complexity [33, 52].

Despite its evolution into a complex code base of over 20 000 lines in recent versions [60], sophisticated exploits continue to bypass verifier checks by leveraging logic flaws in performance optimizations and interface gaps in helper functions [61]. In 2024, over 100 memory related bugs were discovered in the `bpf` subsystem [61]. The verifier has itself become a significant portion of the trusted computing base (TCB), paradoxically introducing the very classes of memory safety vulnerabilities it was designed to prevent.

Recent assessments employing state embedding, which integrates correctness checks into `bpf` algorithms, identified 15 previously undisclosed logic errors within a single month [52]. Two of these were severe enough to allow local privilege escalation, proving that verifier approval is not a guarantee of absolute safety [52]. As developers demand more expressive extensions for cloud-scale environments, the verifier is increasingly forced to use various optimization schemes, which can inadvertently overlook subtle vulnerabilities [33].

To demonstrate the consequences of these failure modes, we focus on a subset of CVEs established by [61] that reveal the limitations of contemporary software-based enforcement mechanisms. For a comprehensive exposition of the underlying taxonomic framework governing these vulnerabilities, we direct the reader to [61].

4.1 Register Dependency Tracking

The eBPF verifier does not analyse all possible execution paths in the program, as discussed in Section 2.2. One of the many strategies uti-

lized during optimisation phases is path pruning [62], which skips redundant execution paths to boost performance. However, sound dependency abstraction between registers is essential for precise path approximation. CVE-2023-2163 [27] shows, that the eBPF verifier monitors such register dependencies improperly during path pruning. This vulnerability enables the attackers to bypass memory safety checks, potentially leading to privilege escalation and arbitrary memory write operations [61].

To exemplify this problem, let us consider R_1 and R_2 registers in a eBPF program [61]. When register R_1 is assigned the result of some pointer arithmetic operation that involves R_2, the verifier should then note that R_1 now depends on R_2. This would then call for the verifier to traverse all possible paths for R_1 affected by R_2, however a PoC from Google security research [63] proves that this is not the case. This leads to the verifier ignoring R_2 as a dependency of R_1 and pruning the aforementioned paths completely. As a result, if R_1 points to an address influenced by an offset R_2, the verifier mistakenly concludes that it is invariably within safe bounds. By running eBPF applications that utilize pointer operations with R_1 and R_2, attackers can manipulate R_1 to reference memory beyond its allocated boundary.

The core problem here is that optimization-driven path pruning can decouple a pointer's actual value from its tracked state. This creates a systemic risk where internal logic flaws allow the execution of unverified, out-of-bounds memory accesses.

4.2 Argument Checking

A primary duty of the eBPF verifier is to ensure that the invocation of helper functions adheres to the pre-defined calling conventions (prototype signatures) [8], however, its implementation is missing certain argument checks which would be vital for memory safety. CVE-2021-4204 [26] demonstrates this lack of argument validation in the verifier for eBPF helper functions, leading to potential out-of-bonds memory violations and similarly to CVE-2023-2163 (4.1); privilege escalation.

As a counterexample, let us consider the implementation of the `bpf_strncmp()` helper function as implemented in the Linux kernel, shown in Listing 1. It accepts a pointer to a memory block as its first argument `ARG_PTR_TO_MEM` and the size of this memory block as its second argument `ARG_CONST_SIZE`. The verifier can then utilize them to validate the correctness of the pointer and the size of the allocated memory to which the pointer refers to.

```
struct bpf_func_proto bpf_strncmp_proto = {
    .func      = bpf_strncmp,
    .gpl_only  = false,
    .ret_type  = RET_INTEGER,
    .arg1_type = ARG_PTR_TO_MEM | MEM_RDONLY,
    .arg2_type = ARG_CONST_SIZE,
    .arg3_type = ARG_PTR_TO_CONST_STR,
};
```

Listing 1: `bpf` function prototype for `strncmp()`.

The root of the vulnerability CVE-2023-2163 are the helper functions `bpf_ringbuf_submit()` and `bpf_ringbuf_discard()`, two functions on the kernel-provided interface for managing ring buffers[2]. The absence of a size parameter in their signatures (Listing 2), as specified by the `bpf` calling convention, renders the verifier ineffective for validating the size of the allocated memory passed to them. In such cases, the verifier will simply skip the otherwise mandatory checks, without rejecting the program or raising a warning. This lack of memory block size validation equips attackers with potential for kernel memory corruption exploits through out-of-bounds pointer passing.

```
struct bpf_func_proto bpf_ringbuf_*_proto = {
    .func      = bpf_ringbuf_*,
    .ret_type  = RET_VOID,
    .arg1_type = ARG_PTR_TO_RINGBUF_MEM | ... ,
    .arg2_type = ARG_ANYTHING,
};
```

Listing 2: Deficient `bpf` function template.

This illustrates a structural vulnerability in the eBPF interface: the safety of memory operations within helpers is predicated on static signatures that may fail to capture or enforce the necessary spatial constraints.

4.3 Analysis Boundary

While the eBPF verifier performs rigorous static analysis on the bytecode, the internal implementations of kernel-provided helper functions are exempt from its scope. As these helpers are pre-compiled into the kernel, they are considered part of the TCB, and the verifier only focuses on the arguments passed to them, ensuring that they conform to their defined prototypes in subsection 4.2. Syzbot [64], the automated fuzzing bot identified temporal safety violations [65] in such helpers like `dev_map_lookup_elem()`, which may return pointers to uninitialized memory.

These helpers will skip memory safety checks and infer that the eBPF application invokes them appropriately, as `dev_map_lookup_elem()` indicates in Listing 3. The `dev_map_lookup_elem()` function retrieves `dev_map` entries without verifying their initialization status. Because the helper

[2] A ring buffer, also known as a circular buffer, is a fixed-size data structure used in many resource constrained environments where static allocation is preferred.

lacks internal checks, the burden of verification shifts to the untrusted eBPF program. This creates a significant security gap, as the verifier currently lacks the semantic awareness to enforce initialization before entry access.

```
void *__dev_map_lookup_elem(
        struct bpf_map *map, u32 key) {
    struct bpf_dtab *dtab = container_of(map,
                                struct bpf_dtab, map);
    struct bpf_dtab_netdev *obj;
    if (key >= map->max_entries)
        return NULL;
    obj = rcu_dereference_check(
            dtab->netdev_map[key],
            rcu_read_lock_bh_held());
    return obj;
}
```

Listing 3: Implementation of `dev_map_lookup_elem`.

Consequently, the safety is limited by a verification gap at the boundary between the eBPF VM and the kernel. Without architectural enforcement, uninitialized or stale data can propagate back into the extended environment without being detected by the static analysis pipeline.

5 Methodology

Our ultimate objective is to offload the burden of safety verification and resolve the tension between performance and safety by moving toward a system where hardware acts as the final arbiter of integrity. Rather than relying on the completeness of a software verifier, our intended solution utilizes the architectural primitives of CHERI capabilities [18] to provide deterministic runtime enforcement. In this model, even if a logic bug allows a malicious program to pass the verifier, any attempt to perform out-of-bounds memory access or pointer forgery results in an immediate, hardware-triggered fault, effectively closing the memory safety gap. The primary goal of this project is to modify the arm64 `bpf` JIT compiler to emit capability-aware instructions, ensuring that every memory access performed by an eBPF program is authorized by a hardware-validated capability.

5.1 Environment Setup

We will utilize the Arm Morello [19] development environment, with the CheriBSD [20] operating system through QEMU [66]. The toolchain will consist of the CHERI-LLVM compiler [21], which supports the `purecap` ABI. We will leverage cheribuild [67] to manage the complex dependencies between the kernel, the LLVM backend, and the QEMU-Morello emulator.

For rapid prototyping and to avoid the constraints of the monolithic Linux kernel, we will conduct our experiments using the `ubpf` [68] library. `ubpf` offers comparable JIT compilation tools to eBPF, but operates in userspace, facilitating simpler navigation while retaining the ability to imitate kernel-like boundaries through its primitives [58].

This step has previously been carried out during our preliminary prototyping efforts.

5.2 JIT Compiler

The core challenge is modifying the JIT backend [9] to transition from 64-bit integer registers (X-registers) to 128-bit capability registers (C-registers). In standard AArch64, eBPF registers $R_0 - R_{10}$ are mapped to native registers $X_0 - X_{10}$. We will redefine this mapping to utilize $C_0 - C_{10}$. This requires updating the JIT compiler register allocation logic to handle the 128-bit width and the implicit tag bit. Every memory access instruction emitted by the JIT (e.g., LDR, STR) must be replaced with its capability equivalent from Table 1 to ensure that the hardware performs bounds and permission checks for every load and store. The JIT must generate a new prologue that initializes the PCC and the DDC. Crucially, the prologue must set the bounds of the context capability C_1 to exactly match the size of the context object, preventing the program from scanning adjacent kernel memory.

5.3 Helper Functions

Helper functions currently expect raw 64-bit pointers. To maintain compatibility, we will implement a capability shim for critical helpers. For helpers like `bpf_map_lookup_elem()`, the shim will intercept the returned raw pointer and wrap it in a capability with bounds derived from the map's `value_size`. We will enforce 16-byte alignment for the internal stack, as required by the Morello hardware for storing capabilities. Any attempt to store a capability at a non-aligned address will trigger a runtime alignment fault.

Additionally, we will need to create fake helper implementations for our evaluation process. Since we deviate from using the Linux eBPF subsystem due to its inherent dependencies on the kernel, pre-compiled helper function implementations will be exempt as well. This independence allows our evaluation to potentially explore asymmetric hardware-software configurations, facilitating experiments with hybrid setups where pure capability-based extensions run atop unhardened kernels, or vice versa.

5.4 Evaluation

To evaluate our solution, we adopt the vulnerability framework established by [61]. While the original work provides an exhaustive enumeration of memory safety failures, we focus on a subset of CVEs that directly exercise the architectural bounds and permissions of the CHERI model. We

plan to address the following sub-research questions to guide our evaluation:

RQ1: Can hardware-enforced capabilities deterministically trap logic vulnerabilities, such as unreliable register tracking [27] and partial argument checking [26], that bypass software-based verification? We define a successful block as the triggering of a hardware capability exception that terminates the eBPF program before an illegal memory access occurs.

RQ2: What are the computational costs associated with transitioning the eBPF JIT from 64-bit integer registers to 128-bit capability-aware primitives in terms of compilation overhead and execution latency?

Exploratory Objective: Beyond our primary research questions, we explore the potential for hardware enforcement to reduce verifier overhead. Specifically, we investigate whether offloading certain safety checks to hardware can permit the safe execution of complex programs that the static verifier would otherwise reject as unprovable or unsafe. While an exhaustive evaluation of all verifier edge cases is beyond our current scope, we provide a preliminary analysis of this trade-off.

The primary risk in our task is the Capability Compression and Alignment requirement. CHERI capabilities are 128-bit primitives that must be naturally aligned; attempts to store or load unaligned capabilities will trigger a hardware failure. eBPF programs, however, frequently perform unaligned memory accesses or partial register spills. Forcing eBPF to adhere to CHERI's alignment may introduce significant performance overhead or break compatibility with existing bytecode. Furthermore, the Pointer Provenance rule poses a risk: if the JIT compiler generates a pointer from arbitrary integer data, the hardware will atomically clear the validity tag, rendering the pointer unusable.

Should the deterministic enforcement of 128-bit capabilities prove too restrictive for existing eBPF bytecode, we will fall back to a Hybrid Isolation Model [22]. In this scenario, we would utilize the DDC to provide coarse-grained spatial safety for the entire eBPF sandbox while relying on the verifier for fine-grained object tracking. If the JIT-level offloading fails to achieve native performance, we will adopt the exception-based point-of-use probing method identified in HIVE [57], where only specific kernel pointers are protected via pointer authentication while standard `bpf` memory remains in a de-privileged userspace-like domain.

Month	Week	Task & Research Phase
May	*P1: Architecture & Environment*	
	18	Finalize study of eBPF CVEs.
	19	Setup Morello toolchain.
	20	Register and memory mapping.
	21	Design JIT capability logic.
	22	BUFFER
June	*P2: JIT Implementation*	
	23	ALU instruction translation.
	24	Memory load/store logic.
	25	Bounds checking integration.
	26	BUFFER
July	*P3: Evaluation & Benchmarking*	
	27	Performance assessment.
	28	Fine-tuning JIT emission.
	29	Morello hardware debugging.
	30	BUFFER
August	*P4: Security Analysis*	
	31	Replicate range analysis CVEs.
	32	Verify spatial safety traps.
	33	Test temporal safety.
	34	Evaluate TCB reduction.
	35	Report on verifier offloading.
Sept	*P5: Finalization*	
	36	Finalize thesis structure.
	37	Draft results chapters.
	38	BUFFER
	39	First draft internal review.
Oct	40	BUFFER
	41	BUFFER
	42	BUFFER
	43	Finalize formatting.
Nov	44–47	Final review and proofreading.
Dec	48	**Submission (Dec 18, 2026)**

Table 2: Weekly project timeline for 2026.

6 Plan

The following timeline outlines the execution of this research project. Given the complexity of implementing a hardware-enforced JIT compiler for the Arm Morello prototype and the constraints of a part-time work schedule, the project is structured into iterative stages spanning from May 2026 to the final submission in December.

The project is divided into five primary phases: *Architecture & Environment*, *JIT Implementation*, *Evaluation & Benchmarking*, *Security Analysis*, and *Finalization*. Each phase corresponds to specific technical milestones and buffer periods to account for the experimental nature of the Morello hardware.

The inclusion of significant buffer periods in the fourth quarter is essential to accommodate hardware-specific debugging on the Morello prototype and the iterative nature of the security analysis under a part-time schedule.

7 Conclusion

The integrity of the Linux kernel increasingly depends on the safety of eBPF-based extensions, yet its current reliance on software-centric static verification has reached a fundamental limit. As program complexity increases, the gap between the abstract models of the verifier and concrete runtime behavior has become a primary attack surface. The persistent discovery of high-severity vulnerabilities partially enumerated in section 4, indicate that the verifier alone is insufficient to guarantee security in the face of modern program complexity [55]. This tension between the need for high-performance modularity and the absolute requirement for kernel safety can no longer be resolved by further hardening of the static analysis engine [54].

This research proposes to delegate the burden of security enforcement from the software verifier to the CHERI microarchitecture. By replacing legacy 64-bit pointers with unforgeable 128-bit capabilities, we transition from a reactive defense model to a hardware-enforced foundation for memory safety [22]. Ultimately, the move toward capability-based eBPF runtimes represents a significant departure from traditional operating system design, aligning kernel extensibility with the principle of least privilege at the instruction level. Delegating bounds enforcement and provenance tracking to the hardware, could potentially eliminate computationally expensive stages of symbolic execution and bytecode preprocessing by significantly trimming the TCB. We suggest, that the architectural overhead of CHERI is a justifiable trade-off for the deterministic security it provides, enabling the Linux kernel to evolve safely into a more modular and robust platform for both cloud-scale and real-time environments.

8 AI statement

During the preparation of this work, Google Gemini was used for text refinement to improve clarity and flow. No other artificial intelligence tools were used for any other purpose. The content was thoroughly reviewed and edited, taking full responsibility for the final outcome.

References

[1] Bolaji Gbadamosi, Luigi Leonardi, Tobias Pulls, *et al.* "The eBPF Runtime in the Linux Kernel." arXiv: 2410.00026. (Sep. 16, 2024), [Online]. Available: http://arxiv.org/abs/2410.00026 (visited on Mar. 12, 2026).

[2] "Building External Modules form The Linux Kernel documentation." (2026), [Online]. Available: https://docs.kernel.org/kbuild/modules.html (visited on Mar. 15, 2026).

[3] Michael M. Swift, Brian N. Bershad, and Henry M. Levy, "Improving the reliability of commodity operating systems," *SIGOPS Oper. Syst. Rev.*, vol. 37, no. 5, pp. 207–222, Oct. 2003, ISSN: 0163-5980. DOI: 10.1145/1165389.945466. [Online]. Available: https://doi.org/10.1145/1165389.945466.

[4] "BPF Instruction Set Architecture (ISA)." (2026), [Online]. Available: https://www.kernel.org/doc/html/latest/bpf/standardization/instruction-set.html (visited on Mar. 15, 2026).

[5] "Classic BPF vs eBPF from The Linux Kernel documentation." (2026), [Online]. Available: https://www.kernel.org/doc/html/latest/bpf/classic_vs_extended.html (visited on Mar. 9, 2026).

[6] Jinghao Jia, Michael V. Le, *et al.*, "Fast (Trapless) Kernel Probes Everywhere," *Preprint*, 2024.

[7] Zicheng Wang, Yicheng Guang, *et al.*, "SeaK: Rethinking the Design of a Secure Allocator for OS Kernel," *Preprint*, 2024.

[8] "Ebpf verifier from the eBPF Documentation." (2026), [Online]. Available: https://docs.ebpf.io/linux/concepts/verifier/ (visited on Mar. 23, 2026).

[9] "Linux kernel jit compilation source for arm64," GitHub. (2026), [Online]. Available: https://github.com/torvalds/linux/blob/master/arch/arm64/net/bpf_jit_comp.c (visited on Mar. 29, 2026).

[10] Yi He and Roland Guo, "Cross Container Attacks: The Bewildered eBPF on Clouds," *Preprint*, 2024.

[11] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, *et al.*, "Simple and precise static analysis of untrusted Linux kernel extensions," in *Proceedings of the 40th ACM SIGPLAN Conference on PLDI*, ACM, Jun. 8, 2019, pp. 1069–1084. DOI: 10.1145/3314221.3314590. [Online]. Available: https://dl.acm.org/doi/10.1

145/3314221.3314590 (visited on Mar. 8, 2026).

[12] Soo Yee Lim, Xueyuan Han, and Thomas Pasquier, "Unleashing Unprivileged eBPF Potential with Dynamic Sandboxing," in *Proceedings of the 1st Workshop on eBPF*, ACM, Sep. 2023, pp. 42–48. DOI: 10.1145/3609021.3609301. [Online]. Available: https://dl.acm.org/doi/10.1145/3609021.3609301 (visited on Feb. 19, 2026).

[13] "Armv8.1-M PACBTI Extensions." (2026), [Online]. Available: https://developer.arm.com/documentation/109576/0100/Pointer-Authentication-Code/Introduction-to-PAC (visited on Mar. 15, 2026).

[14] Soo Yee Lim, Tanya Prasad, Xueyuan Han, and Thomas Pasquier. "SafeBPF: Hardware-assisted Defense-in-depth for eBPF Kernel Extensions." arXiv: 2409.07508. (Sep. 11, 2024), [Online]. Available: http://arxiv.org/abs/2409.07508 (visited on Feb. 10, 2026).

[15] "MTE User Guide for Android OS." (2026), [Online]. Available: https://developer.arm.com/documentation/108035/0100/Introduction-to-the-Memory-Tagging-Extension (visited on Mar. 15, 2026).

[16] Lei Song and Jiaxin Li, "eBPF: Pioneering Kernel Programmability and System Observability," in *2024 3rd AICIT*, Sep. 2024, pp. 1–10. DOI: 10.1109/AICIT62434.2024.10730620. [Online]. Available: https://ieeexplore.ieee.org/document/10730620/ (visited on Mar. 15, 2026).

[17] Juhee Kim *et al.*, "Tiktag: Breaking ARM's Memory Tagging Extension with Speculative Execution," in *2025 IEEE Symposium on Security and Privacy (SP)*, May 2025, pp. 4063–4081. DOI: 10.1109/SP61157.2025.00039. [Online]. Available: https://ieeexplore.ieee.org/document/11023460/ (visited on Feb. 6, 2026).

[18] Robert N. M. Watson, Peter G. Neumann, Jonathan Woodruff, *et al.*, "Capability Hardware Enhanced RISC Instructions: CHERI Instruction-Set Architecture (Version 9)," University of Cambridge. DOI: 10.48456/TR-987. [Online]. Available: https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-987.html (visited on Feb. 28, 2026).

[19] "Morello Platform Open Source Software." (2026), [Online]. Available: https://www.morello-project.org/ (visited on Mar. 15, 2026).

[20] "CheriBSD." (2026), [Online]. Available: https://www.cheribsd.org/ (visited on Mar. 29, 2026).

[21] *CTSRD-CHERI/llvm-project*, Capability Hardware Enhanced RISC Instructions, Mar. 13, 2026. [Online]. Available: https://github.com/CTSRD-CHERI/llvm-project (visited on Mar. 29, 2026).

[22] Robert N.M. Watson, Jonathan Woodruff, *et al.*, "CHERI: A Hybrid Capability-System Architecture for Scalable Software Compartmentalization," in *2015 IEEE Symposium on Security and Privacy*, IEEE, May 2015, pp. 20–37. DOI: 10.1109/SP.2015.9. [Online]. Available: https://ieeexplore.ieee.org/document/7163016/ (visited on Feb. 28, 2026).

[23] Soo Yee Lim, Sidhartha Agrawal, Xueyuan Han, *et al.* "Securing Monolithic Kernels using Compartmentalization." arXiv: 2404.08716. (Apr. 12, 2024), [Online]. Available: http://arxiv.org/abs/2404.08716 (visited on Feb. 19, 2026).

[24] Hao Sun and Zhendong Su, "Prove It to the Kernel: Precise Extension Analysis via Proof-Guided Abstraction Refinement," in *Proceedings of the ACM SIGOPS 31st SOSP*, ACM, Oct. 13, 2025, pp. 736–751. DOI: 10.1145/3731569.3764796. [Online]. Available: https://dl.acm.org/doi/10.1145/3731569.3764796 (visited on Mar. 15, 2026).

[25] Nathaniel Wesley Filardo *et al.*, "Cornucopia Reloaded: Load Barriers for CHERI Heap Temporal Safety," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, La Jolla CA USA: ACM, Apr. 27, 2024, pp. 251–268, ISBN: 979-8-4007-0385-0. DOI: 10.1145/3620665.3640416. [Online]. Available: https://dl.acm.org/doi/10.1145/3620665.3640416 (visited on Apr. 7, 2026).

[26] "NVD - CVE-2021-4204." (2026), [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2021-4204 (visited on Mar. 28, 2026).

[27] "NVD - CVE-2023-2163." (2026), [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2023-2163 (visited on Mar. 28, 2026).

[28] "BPF Documentation from The Linux Kernel documentation." (2026), [Online]. Available: https://docs.kernel.org/bpf/ (visited on Mar. 8, 2026).

[29] "What is eBPF? An Introduction and Deep Dive." (2026), [Online]. Available: https://ebpf.io/what-is-ebpf/ (visited on Mar. 12, 2026).

[30] "Linux/Documentation/Bpf at master - torvalds/linux." (2026), [Online]. Available: https://github.com/torvalds/linux/blob/master/Documentation/bpf/classic_vs_extended.rst (visited on Mar. 12, 2026).

[31] "Llvm-project bpf backend target," GitHub. (2026), [Online]. Available: https://github.com/llvm/llvm-project/blob/main/llvm/lib/Target/BPF (visited on Mar. 29, 2026).

[32] "eBPF verifier from The Linux Kernel documentation." (2026), [Online]. Available: https://docs.kernel.org/bpf/verifier.html (visited on Mar. 23, 2026).

[33] Jinsong Mao, Hailun Ding, Juan Zhai, and Shiqing Ma, "Merlin: Multi-tier Optimization of eBPF Code for Performance and Compactness," in *Proceedings of ASPLOS 2024*, ACM, Apr. 27, 2024, pp. 639–653. DOI: 10.1145/3620666.3651387. [Online]. Available: https://dl.acm.org/doi/10.1145/3620666.3651387 (visited on Mar. 12, 2026).

[34] "Helper Function bpf_tail_call from eBPF Documentation." (2026), [Online]. Available: https://docs.ebpf.io/linux/helper-function/bpf_tail_call/ (visited on Mar. 29, 2026).

[35] Liz Rice, *Learning eBPF: Programming the Linux Kernel for Enhanced Observability, Networking, and Security*, First edition. Beijing Boston Farnham Sebastopol Tokyo: O'Reilly, 2023, 217 pp., ISBN: 978-1-0981-3512-6.

[36] "Bpf-helpers(7) from Linux manual page." (2026), [Online]. Available: https://www.man7.org/linux/man-pages/man7/bpf-helpers.7.html (visited on Mar. 29, 2026).

[37] "Helper Function bpf_map_lookup_elem from eBPF Documentation." (2026), [Online]. Available: https://docs.ebpf.io/linux/helper-function/bpf_map_lookup_elem/ (visited on Mar. 29, 2026).

[38] "Helper Function bpf_redirect from eBPF Documentation." (2026), [Online]. Available: https://docs.ebpf.io/linux/helper-function/bpf_redirect/ (visited on Mar. 29, 2026).

[39] "Helper Function bpf_trace_printk from eBPF Documentation." (2026), [Online]. Available: https://docs.ebpf.io/linux/helper-function/bpf_trace_printk/ (visited on Mar. 29, 2026).

[40] "Ebpf trampolines from eBPF Documentation." (), [Online]. Available: https://docs.ebpf.io/linux/concepts/trampolines/ (visited on Mar. 29, 2026).

[41] Robert N. M. Watson, Simon W. Moore, *et al.*, "An Introduction to CHERI," *Technical Report*, 2024.

[42] Jack B. Dennis and Earl C. Van Horn, "Programming Semantics for Multiprogrammed Computations," *ACM Communications*, 1966.

[43] Jonathan Woodruff, Robert N. M. Watson, *et al.*, "The CHERI capability model: Revisiting RISC in an age of risk," *Preprint*, 2024.

[44] Elliott Irving Organick, *Computer system organization: The b5700/b6700 series (acm monograph series)*, 1973.

[45] Maurice Vincent Wilkes and Roger Michael Needham, *The cambridge cap computer and its operating system*, 1979.

[46] Henry M. Levy, *Capability-Based Computer Systems*. Digital Press, May 16, 2014, ISBN: 978-1-4831-0106-4.

[47] Ilya Sergey, Ed., *Programming Languages and Systems*. Springer, 2022. DOI: 10.1007/978-3-030-99336-8. [Online]. Available: https://link.springer.com/10.1007/978-3-030-99336-8 (visited on Mar. 26, 2026).

[48] Brooks Davis, Robert N. M. Watson, *et al.*, "CheriABI: Enforcing Valid Pointer Provenance in the POSIX C Run-time," in *Proceedings of ASPLOS 2019*, ACM, Apr. 4, 2019, pp. 379–393. DOI: 10.1145/3297858.3304042. [Online]. Available: https://dl.acm.org/doi/10.1145/3297858.3304042 (visited on Mar. 27, 2026).

[49] "Linux kernel bpf verifier source." (2026), [Online]. Available: https://github.com/torvalds/linux/blob/master/kernel/bpf/verifier.c (visited on Mar. 29, 2026).

[50] Jinghao Jia, Raj Sahu, *et al.*, "Kernel extension verification is untenable," in *Proceedings of the 19th HotOS*, ACM, Jun. 22, 2023, pp. 150–157. DOI: 10.1145/3593856.3595892. [Online]. Available: https://dl.acm.org/doi/10.1145/3593856.3595892 (visited on Mar. 15, 2026).

[51] Yifei Wu, Qirui Liu, Wenbo Shen, *et al.*, "Flow Hijacking in eBPF: Exploitation and Mitigation Across both Interpreter and JIT Execution," *International Journal of Information Security*, vol. 24, no. 5, p. 204, Oct. 2025. DOI: 10.1007/s10207-025-01124-x. [Online]. Available: https://link.spring

er.com/10.1007/s10207-025-01124-x (visited on Feb. 12, 2026).

[52] Hao Sun and Zhendong Su, "Validating the eBPF Verifier via State Embedding," *Preprint*, 2024.

[53] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. "Sound, Precise, and Fast Abstract Interpretation with Tristate Numbers." arXiv: 2105.05398. (Dec. 15, 2021), [Online]. Available: http://arxiv.org/abs/2105.05398 (visited on Mar. 15, 2026).

[54] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte, "Verifying the Verifier: eBPF Range Analysis Verification," in *Computer Aided Verification*, vol. 13966, Springer Nature, 2023, pp. 226–251. DOI: 10.1007/978-3-031-37709-9_12. [Online]. Available: https://link.springer.com/10.1007/978-3-031-37709-9_12 (visited on Mar. 15, 2026).

[55] Luis Gerhorst, Henriette Herzog, *et al.*, "VeriFence: Lightweight and Precise Spectre Defenses for Untrusted Linux Kernel Extensions," in *The 27th International Symposium on RAID*, ACM, Sep. 30, 2024, pp. 644–659. DOI: 10.1145/3678890.3678907. [Online]. Available: https://dl.acm.org/doi/10.1145/3678890.3678907 (visited on Mar. 15, 2026).

[56] Hongyi Lu, Shuai Wang, *et al.*, "MOAT: Towards Safe BPF Kernel Extension," *Preprint*, 2024.

[57] Peihua Zhang, Chenggang Wu, *et al.*, "HIVE: A Hardware-assisted Isolated Execution Environment for eBPF on AArch64," *Preprint*, 2024.

[58] Paul Metzger, A Theodore Markettos, Edward Tomasz Napierała, Matthew Naylor, Robert N M Watson, and Timothy M Jones, "Deprivileging Low-Level GPU Drivers Efficiently with User-Space Processes and CHERI Compartments," *Preprint*, 2025.

[59] Chathuranga Sampath Kalutharage, Saket Mohan, Xiaodong Liu, and Christos Chrysoulas, "Enhancing Automotive Intrusion Detection Systems with Capability Hardware Enhanced RISC Instructions-Based Memory Protection," *Electronics*, vol. 14, no. 3, p. 474, Jan. 24, 2025. DOI: 10.3390/electronics14030474. [Online]. Available: https://www.mdpi.com/2079-9292/14/3/474 (visited on Mar. 8, 2026).

[60] Paul Chaignon. "BPF Verifier State Pruning: Timeline." (Jan. 20, 2026), [Online]. Available: https://pchaigno.github.io/ebpf/2026/01/20/state-pruning-timeline.html (visited on Mar. 9, 2026).

[61] Kaiming Huang, Mathias Payer, and Zhiyun Qian, "SoK: Challenges and Paths Toward Memory Safety for eBPF," *Preprint*, 2024.

[62] "eBPF verifier path pruning." (2026), [Online]. Available: https://docs.kernel.org/bpf/verifier.html#pruning (visited on Mar. 28, 2026).

[63] "Security-research/Pocs/Linux/Cve-2023-2163." (2026), [Online]. Available: https://github.com/google/security-research/tree/master/pocs/linux/cve-2023-2163 (visited on Mar. 28, 2026).

[64] "Syzbot official website (upstream)." (2026), [Online]. Available: https://syzbot.org/upstream (visited on Mar. 28, 2026).

[65] "KMSAN: Uninit-value in ebpf function (syzbot)." (2026), [Online]. Available: https://syzkaller.appspot.com/bug?extid=1a3cf6f08d68868f9db3 (visited on Mar. 28, 2026).

[66] "QEMU." (), [Online]. Available: https://www.qemu.org/ (visited on Apr. 18, 2026).

[67] *CTSRD-CHERI/cheribuild*, Capability Hardware Enhanced RISC Instructions, Mar. 16, 2026. [Online]. Available: https://github.com/CTSRD-CHERI/cheribuild (visited on Mar. 29, 2026).

[68] *Iovisor/ubpf*, Mar. 15, 2026. [Online]. Available: https://github.com/iovisor/ubpf (visited on Mar. 15, 2026).